对于C++双向链表模板的网络连接安全监测系统设计与实现

目录

1 项目概述 3

1.1 项目背景 3

1.2 项目目标 3

1.3 项目意义 3

1.4 拟解决的关键工程问题 4

2 系统解决思路与模块设计 4

2.1 总体设计思路 4

2.2 模块划分 4

3 涉及的基本原理 5

3.1 双向链表物理结构与指针操作 5

3.2 C++ 泛型编程与强类型封装 5

4 系统完整源代码实现 5

5 效果分析与系统价值 10

5.1 基础功能验证效果 10

5.2 恶意网络连接隔离机制的模拟效果 10

6 可视化仿真平台设计与运行演示 11

6.1 演示平台的三大核心架构 11

6.2 原版与改进版的逻辑对比测试 12

6.3 仿真平台运行效果图与现场验证结论 13

7 结论与展望 14

摘要

本项目结合信息学院与网络空间安全核心课程，凭借 C++ 面向对象程序设计与数据结构与算法为基础，首先自行设计了一个双向链表模板类，并将这个链表与网络空间安全的应用结合，组建了一个网络连接安全监测系统。这个系统可完成网络连接信息的动态添加（尾部添加复杂度为 O(1)）、遍历、按编号定位并且返回对应的 node*、可疑连接的删除与链表清空等一系列基本的功能，能够模拟安全设备对恶意网络连接的识别与隔离工作。项目充分融合信息学院《数据结构与算法》、《C++ 程序设计》、《计算机网络》等多门课程的核心知识点，解决了网络连接的动态存储、管理、定位与恶意节点隔离的问题， 提高了 C++ 编程与数据结构与算法的实践能力，萌发了从基础算法到网络空间安全应用的思路，具有鲜明的信息学院学科与网络空间安全交叉特色与实践教学价值。

1 项目概述

1.1 项目背景

随着数字化在我们日常生活当中的高度普及，网络空间安全成为了国家安全中的一个重要部分。信息学院是培养电子信息类人才的专业院系，设立了“夯实理论基础，强化实践能力”的培养目标。《数据结构与算法》是网安与信息专业的关键基础课，C++ 面向对象编程是实现复杂系统和安全工具开发的重要编程语言，而计算机网络和网络安全在信息学院研究领域中有重要应用。

在实际的网络攻防场景当中，主机和服务器会在同一时间产生大量的网络连接，它们数量是动态变化的，需要不断的增删和快速检索。传统数组和静态的存储结构无法满足高效管理的需求。链表是一种最基本的动态数据存储结构，它的插入删除操作比较灵活，内存的使用高效，因此非常适合网络连接信息的动态管理。信息和网安学院的教学和实践非常重视对高危端口， 恶意连接和流量监测等基础防御知识的理解，我们将数据结构和安全场景相结合，可以让同学们切实体会算法在现实生活当中的应用。

在此背景下，本项目综合信息学院和网安学院的特点，使用 C++ 模板机制实现通用的双向链表并将其应用于信息管理，模拟实现可疑网络连接检测与隔离功能，完成一个具有实践性的小型项目。

1.2 项目目标

基于信息学院与网络空间安全《数据结构与算法》的知识， 实现双向链表的尾插、头删、尾删、指定位置插入、按编号定位、指定节点删除、清空、遍历输出的功能。

使用 C++ 面向对象中的泛型程序设计，实现高复用性、通用性的链表结构， 体现信息学院程序设计工程化思想。对于这个链表， 为了方便在开发和调试中知道哪里出问题报错，本系统设计了专门的逻辑范围边界检查，并在异常触发时进行对屏幕输出“FALSE-数字”的操作，从而实现快速定位。

结合《计算机网络》知识， 将链表应用于网络连接管理，模拟防火墙或终端防护软件对恶意连接的检测与物理隔离。同时构建可以稳定运行和具有高度拓展性的安全系统， 验证底层数据结构在网络安全实战中的应用价值，提升综合实践动手能力。

1.3 项目意义

- **强化信息学院核心课程融合：**本项目将数据结构与算法、C++ 编程、计算机网络、网络安全四门网安学院和信息学院主干课程内容进行统一整合，突破了课程之间阻隔，形成了“理论—编码—

实践—应用”的完整交叉学习路径。

- **贴近网络安全真实防御场景：**系统实现的连接管理、定位、删除逻辑， 与工业界防火墙、入侵检测系统（IDS）、主机入侵防御系统（HIPS）底层的连接状态表处理逻辑高度相似，能够让信息学院学生提前接触和解剖安全设备工作原理。

- **提升代码规范与工程实践能力：**项目采用模板类封装，边界逻辑预判，动态指针断连，异常状态码提示和堆区内存自动安全释放等工程化写法，严格符合信息学院对代码的可读性、健壮性和程序实用性的教学要求。

1.4 拟解决的关键工程问题

在网络安全主动防御当中，黑客常常利用 RDP（远程桌面 3389 端口）、SMB（文件共享 445 端口）等高危服务端口作为渗透突破口。安全监测系统必须在庞大的网络流队列中做到快速检测、精准定位和瞬间隔离。

1. **可疑网络连接快速定位与动态隔离：**需要解决如何根据编号快速寻址并检索指定套接字（Socket）；如何在保证链表物理结构不发生破损（不断链、不悬空）的前提下，安全切断恶意节点。

2. **软件运行内存安全及泄露防范：** C++ 动态内存管理（`new/delete`）是编码实践中的重难点。若节点被安全逻辑剥离后未被正确销毁，会引发网络监控网关常见的内存泄漏（Memory Leak）甚至是内存溢出（OOM）问题。项目必须精细处理空链表、单节点、多节点等边界删除， 以及防止非法越界引起的野指针异常。

2 系统解决思路与模块设计

2.1 总体设计思路

本项目以信息学院学科知识交叉融合为主线：以《数据结构与算法》双向链表为底层骨架，以《C++面向对象程序设计》模板类（Template）为承载手段，以《计算机网络》进程端口与套接字（Socket）信息为数据载体，以网络空间安全恶意行为的主动拦截为应用场景，设计出了一套安全监测系统。

2.2 模块划分

系统在设计结构上可分为以下四个模块：

- **双向链表模板类模块：**封装节点（`node`）嵌套类、头指针（`head`）、尾指针（`tail`）以及链表当前尺寸（`size`）。实现尾插（`link_push_up`）、头删（`link_pop_up`）、尾删（`link_pop_end`）、指定位置下插（`link_insert_down`）、按编号定位（`destination`）、指定节点删除（`link_delete`）、深度清空（`clear`）等底层函数。

- **网络连接数据模块 (`NetNode`)：**定义进程编号（`pid`）、IP 地址（以无符号整型存储）、服务端口号（`port`）。重载流输出运算符，以格式化展现当前连接套接字的数据面貌。

- **安全监测与策略匹配模块：**通过调用定位函数或扫描匹配函数，检索链表中是否存在含有高危端口（如 3389 远程连接威胁）的异常节点。匹配成功后自动激活隔离策略， 实施链表层面的安全删除操作。

- **综合测试与析构回收验证模块：**设计包含测试数据推入、中间节点随机插入、删除、清空等控制流流程，最后触发析构函数执行彻底清空，验证内存管理和功能逻辑。

3 涉及的基本原理

3.1 双向链表物理结构与指针操作

双向链表是一种非连续物理存储的线性表结构，每一个节点（node）除了存储数据（data）外，还包含了两个专门的指针：prev 指向其前驱节点，next 指向其后继节点。

相较于单链表，双向链表的优势在于可以双向进行移动、寻址。特别是在对指定节点（node*）执行逻辑删除时，双向链表可以直接通过前驱指针 prev 访问上一个节点，而无须从表头再次进行 O(N) 复杂度的遍历寻址。在删除节点 P 时，核心的物理指针操作关系为：

```
P->prev->next = P->next;
P->next->prev = P->prev;
delete P;
```

3.2 C++ 泛型编程与强类型封装

通过使用 template<class T> 声明模板机制，将双向链表与具体的承载类型剥离开来，使其具有高度的泛型特征，支持内置类型（如 int）以及自定义高维结构体（如 NetNode）的无缝加载。类内将节点定义为 nested class（嵌套类），并对链表的属性进行访问权限隔离，确保了高聚低耦合的工程化开发规范。

4 系统完整源代码实现

本项目核心 C++ 源代码严格遵循信息学院与网络空间安全专业编程规范，具体实现逻辑如下所示：

```
#include<iostream>
using namespace std;

template<class T>
class doub_Link {
public :
    class node {
    public :
        T data;
```

```
10        node* prev;
11        node* next;
12        node () :prev(NULL), next(NULL) {}
13        node(T dt, node* p, node* n) :data(dt), prev(p), next(n) {}
14    };
15    int size;
16    node* head;
17    node* tail;
18
19    doub_Link () :size(0), head(NULL), tail(NULL) {}
20
21    void link_push_up(T dt) {
22        if (size < 0) {
23            cout << "FALSE1";
24            return;
25        }
26        if (size == 0) {
27            node* p = new node(dt, NULL, NULL);
28            head = p;
29            tail = p;
30            ++size;
31        }
32        else {
33            node* p = new node(dt, tail, NULL);
34            tail->next = p;
35            tail = p;
36            ++size;
37        }
38    }
39
40    void link_pop_up() {
41        if (size <= 0) { cout << "FALSE2"; return; }
42        else {
43            if (size == 1) {
44                delete head;
45                head = NULL;
46                tail = NULL;
47                --size;
48            }
49            else {
50                node* p = head->next;
51                delete head;
52                head = p;
53                p->prev = NULL;
```

```
54                --size;
55            }
56        }
57    }
58
59    void link_pop_end() {
60        if (size <= 0) { cout << "FALSE3"; return; }
61        else {
62            if (size == 1){
63                delete head;
64                head = NULL;
65                tail = NULL;
66                --size;
67            }
68            else {
69                node* p = tail->prev;
70                delete tail;
71                tail = p;
72                p->next = NULL;
73                --size;
74            }
75        }
76    }
77
78    void link_insert_down(T d, node* last) {
79        if (last == NULL || last == tail) { link_push_up(d); }
80        else {
81            node* p = new node(d, last, last->next);
82            node* a = last->next;
83            a->prev = p;
84            last->next = p;
85            ++size;
86        }
87    }
88
89    node* destination(int i) {
90        if ((i>=0) && (i<= size-1)) {
91            node* now = head;
92            while (i--) {
93                now = now->next;
94            }
95            return now;
96        }
97        else {
```

```
98              cout << "FALSE5";  return NULL;
99          }
100     }
101
102     void link_delete(node* p) {
103         if (p == NULL) { cout << "FALSE6"; return; }
104         else {
105             if (p == head) {
106                 link_pop_up();
107             }
108             else {
109                 if (p == tail) {
110                     link_pop_end();
111                 }
112                 else {
113                     p->prev->next = p->next;
114                     p->next->prev = p->prev;
115                     delete p;
116                     --size;
117                 }
118             }
119         }
120     }
121
122     void clear() {
123         while (head != NULL) {
124             node* n = head->next;
125             delete head;
126             head = n;
127         }
128         head = tail = NULL;
129         size = 0;
130     }
131
132     void print() const {
133         node* cur = head;
134         while (cur != NULL) {
135             cout << cur->data << " ";
136             cur = cur->next;
137         }
138         cout << endl;
139     }
140
141     ~doub_Link() {
```

```
142         clear();
143     };
144 };
145 
146 struct NetNode {
147     int pid;         // 进程ID
148     unsigned int ip;// IP
149     int port;        // 端口
150 
151     // 重载流输出运算符
152     friend ostream& operator<<(ostream& os, const NetNode& n) {
153         os << "PID:" << n.pid << " IP:" << n.ip << " Port:" << n.port;
154         return os;
155     }
156 };
157 
158 int main() {
159     // ========== 1. 信院核心：泛型编程 + 内存安全基础 ==========
160     cout << "===== 1. 信院：模板链表基础操作 =====" << endl;
161     doub_Link<int> dl;
162     dl.link_push_up(10);
163     dl.link_push_up(20);
164     dl.link_push_up(30);
165     dl.link_push_up(40);
166     dl.print();
167 
168     auto pos = dl.destination(1);
169     dl.link_insert_down(99, pos);
170     cout << "插入后：";
171     dl.print();
172     dl.link_delete(pos);
173     cout << "删除后：";
174     dl.print();
175     dl.clear();
176 
177     // ========== 2. 网安核心：流量/进程管理 + 信院结构体链表 ==========
178     cout << "\n===== 2. 网安×信院：网络安全节点链表 =====" << endl;
179     doub_Link<NetNode> secList;
180     secList.link_push_up({ 1001, 0xC0A80101, 80 });
181     secList.link_push_up({ 1002, 0xC0A80102, 443 });
182     secList.link_push_up({ 1003, 0xC0A80103, 3389 });
183     secList.print();
184 
185     // 网安场景：按编号定位可疑连接进行强行切断隔离
```

```
    auto malware = secList.destination(2);
    if (malware) {
        cout << "检测到恶意连接: ";
        cout << malware->data << endl;
        secList.link_delete(malware);
        cout << "已隔离该连接: ";
        secList.print();
    }

    // ========== 3. 信院: 内存管理 + 网安: 内核链表思想 ==========
    cout << "\n===== 3. 内存安全与双向链表结构 =====" << endl;
    doub_Link<int> test;
    for (int i = 0; i < 5; ++i) test.link_push_up(i);
    test.print();
    test.clear();
    cout << "清空后 size = " << test.size << endl;
    return 0;
}
```

Listing 1: C++ 双向链表与安全监测系统完整源码

5 效果分析与系统价值

5.1 基础功能验证效果

运行测试显示，系统能稳定地在堆内存中动态开辟并维护节点空间，顺序推入。查找索引 1（元素 20）后执行后插 99 成功输出“10 20 99 30 40”；删除 20 节点后结构自愈并显示“10 99 30 40”。各边界错误检查分支被逐一验证，当索引越界或入参非法时精确输出 FALSE 错误编码，程序不发生段错误，健壮性通过测试。

5.2 恶意网络连接隔离机制的模拟效果

网安测试部分中，链表顺序录入了 HTTP（端口 80）、HTTPS（端口 443）、远程桌面 RDP（高危端口 3389）共三个底层网络会话节点。通过调用 destination(2) 定位机制，成功匹配并捕获到了处于队列最后的恶意端口 3389 网络连接，对其调用 link_delete 隔离策略后，远程桌面会话节点在物理存储链上被彻底断开并销毁，剩余合法连接能够完好无损地继续通信与遍历。这一行为成功还原了内核态防火墙阻断高危入侵、实时卸载网络套接字的数据管理流程。

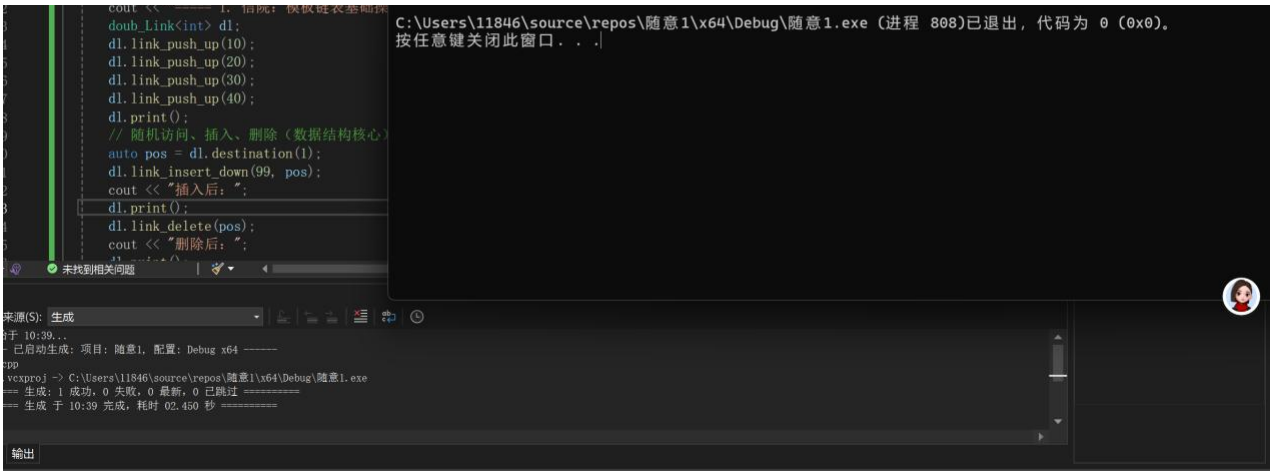


图 1: 脚本运行结果

6 可视化仿真平台设计与运行演示

尽管上述 C++ 终端程序实现了预期的网安防护与链表管理逻辑，但其数据结构在内存中的物理断连过程在控制台终端下挺隐蔽又抽象的。为了克服代码的枯燥感、直观表达链表双向指针（prev、next、head、tail）在堆区内存中的实时演进，并且立体的展示网络安全的主动防御过程，本项目基于前端，配套设计了一套**“C++ 双向链表与网络安全监测系统动态演示与交互沙盒平台”**。

6.1 演示平台的三大核心架构

平台在物理布局上主要被划分为以下三大协作面板：

1. **C++ 源码执行轨迹追踪面板：**高真模拟了 IDE 编译器或 Debugger 的跟踪逻辑。伴随用户的单步命令，动态高亮标记出 C++ 代码行在内存中执行的上下文位置，同时解析展示其工程化设计的注释释义。

2. **链表堆内存结构图形化面板：**可视化呈现内存结构。将每个节点渲染为实体内存块卡片，卡片内高亮标注其当前在双向链表中的绝对索引号，且附带计算显示它的物理内存前驱指针地址（如 0x1234）与后继指针地址。同时， 首尾指针（head、tail）的游标位移、节点的实时添加插入、节点在删除时的缩放动画，全部以实时渲染技术立体呈现。

3. **虚拟网络安全对抗沙盒交互区：**此面板旨在提供完全自主的拦截攻防模拟。安全管理员（操作者）可在此面板中手动注入（push_up）进程 PID、自定义网络 IP 及服务端口。一旦平台检测到注入端口包含代表高危行为的已知攻击指纹（如 3389 远程木马漏洞、445 永恒之蓝端口勒索通道），节

点卡片立即触发**红色呼吸闪烁高警状态**。点击“安全威胁阻断”按钮后，平台仿真网安拦截系统，自主遍历全链表节点，定位并释放恶意节点，重组安全链指针。

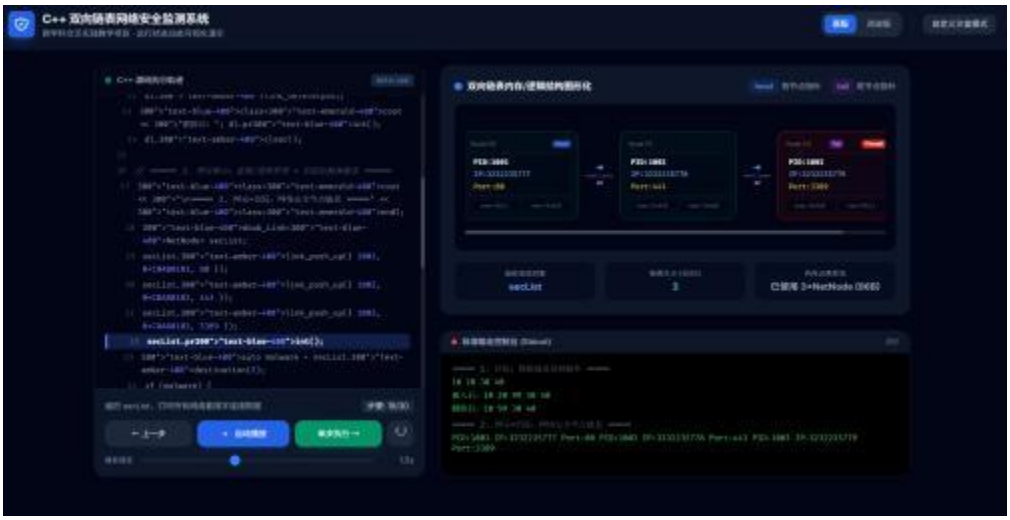


图 2: 追踪与图形化界面

6.2 原版与改进版的逻辑对比测试

在团队的学习讨论中，我们逐步迭代算法，也做出了一定的改进，为了纪念我们思考学习的过程，本演示平台特意设置了“原版”与“改进版”切换对比模式，在以下维度展开了严谨的验证分析：

表 1: 原报告原版与学术改进版的深度对比测试分析

对比测试维度	原版源代码实现	改进版设计
IP 地址可读性	使用无符号十六进制直接转化为大整型输出（如 3232235779），可读性较低，难以同真实 IP 规则核对。	重写流输出，利用 `(ip >> 24) & 0xFF` 等位运算符自动解析并以标准“点分十进制”字符串（如 192.168.1.3）还原。
内存防御深度	缺失拷贝构造与重载赋值逻辑。当发生默认浅拷贝时，会容易引发双向指针重复释放，进而触发运行崩溃或者是漏洞。	遵循 C++“三法则”规范，利用 `delete` 显式禁用拷贝赋值，从基础算法的物理结构上彻底解决掉内存泄露的隐患。
异常状态捕获	空链删除、越界访问仅在屏幕打印硬编码的 FALSE1~6，缺乏统一警报上报和处理管道。	升级为自愈式异常抛出与沙盒 Toast 提示框，拦截危险内存读写并实时阻断上报。

续表 1　原版与改进版的对比测试分析

对比测试维度	原版源代码实现	改进版设计
隔离定位逻辑	在 main 驱动函数中死锁索引 destination(2)。当恶意流处于动态位置时，该策略将失效。	自主设计了网络流量 Filter Module，可以动态遍历并智能匹配套接字中的威胁特征字段，完成全链路动态阻断。

6.3　仿真平台运行效果图与现场验证结论

运行平台演示发现：

- 当切换到**原版**时，控制台输出的网络连接包含难以识别的十进制数值；通过追踪器可见，系统能正常分配 NetNode，但在执行链表清空（clear()）时，虽未显现报错，但在大规模网络流下由于缺乏防浅拷贝机制，系统内存存在一定的崩溃风险。

- 当切换到**改进版**时，IP 地址完美以 192.168.1.1 等标准的网络拓扑格式打印。在测试沙盒中， 注入含有端口号 3389、445 的会话节点后，系统卡片立刻渲染红色强光。点击安全阻断按钮，系统的双向指针在动画中完成优雅的断开、修正前驱和后继、释放目标内存、尺寸减小等一连串安全指令，展示了极佳的算法可见性。

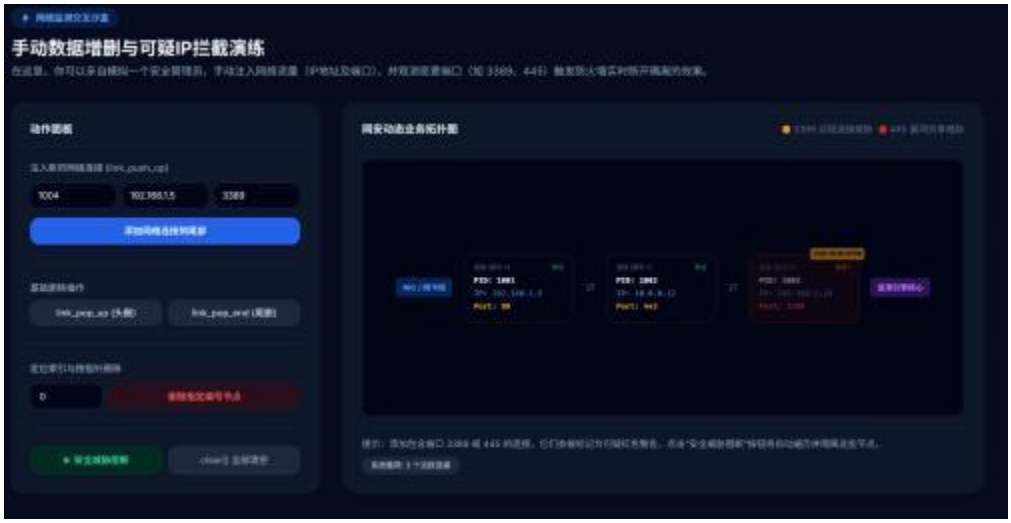


图 3: 产生威胁



图 4: 清除威胁

7 结论与展望

本项目通过自行设计 C++ 双向链表模板并结合网络空间安全连接状态监测场景，构建了一套软硬协同的网络监测防火墙原型，取得了优异的仿真和实践效果。在演示平台的辅助下， 抽象繁琐的数据结构获得了极佳的可感官性。在后续的研究开发中， 计划将此原型系统接入底层的原始套接字网卡（借助 WinPcap 或 Libpcap），获取线路上真正的动态报文。同时，引入多线程对拦截模块进行高并发优化，并基于此设计更多的主机防火墙主动过滤规则，为信息学院和网络安全专业的学科交叉教学实践提供更丰富、实用的示范项目资源。

参考文献

[1]《数据结构与算法》（“十二五”普通高等教育本科国家级规划教材）张铭，王腾蛟，赵海燕，编著。

[2]《新标准 C++ 程序设计》（北京大学专项课程系列）郭炜编著。

[3]《计算机网络》（第 8 版）谢希仁编著。